Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 7:
Software Defined Networking con Mininet

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'Hands-On 7 introduce il paradigma del **Software Defined Networking (SDN)** tramite l'emulatore **Mininet**. L'obiettivo dell'esercitazione è duplice: da un lato, comprendere la virtualizzazione delle risorse di rete tramite i Network Namespaces di Linux; dall'altro, progettare topologie personalizzate in Python capaci di emulare scenari reali. Il sistema deve permettere la configurazione di un router software per l'instradamento tra subnet e l'applicazione di vincoli fisici (*bandwidth* e *delay*) sui link virtuali per testare la robustezza dei protocolli in condizioni non ideali.

2 Stato dell'Arte

Il paradigma SDN rivoluziona le reti tradizionali separando il piano di controllo dal piano dei dati.

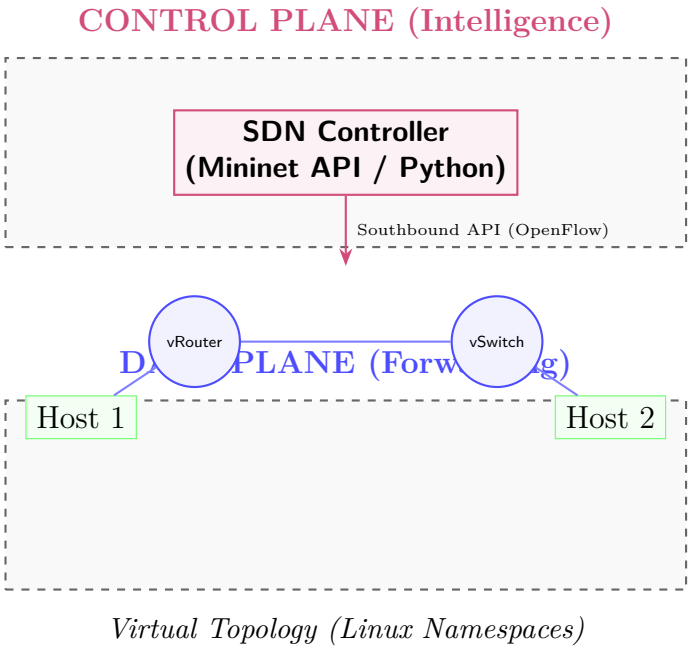

Figure 1: Architettura SDN: Separazione tra logica di controllo centrallizzata e piano dati virtualizzato.

- **Control Plane vs Data Plane**: In una rete SDN, l'intelligenza (routing, policy) è centralizzata in un controller software, mentre gli switch si limitano a inoltrare i pacchetti seguendo le istruzioni ricevute.
- **Mininet**: È l'emulatore standard de-facto per la ricerca SDN. Utilizza i container leggeri del kernel Linux per creare host, switch e router su una singola macchina, garantendo un isolamento perfetto delle interfacce.
- **Traffic Control (TC)**: Linux fornisce il sottosistema `tc` per gestire il queuing e la limitazione del traffico. Mininet espone questa potenza tramite la classe `TCLink`, permettendo di simulare latenze geografiche e limiti di throughput.

3 Metodologia ed Implementazione

L'attività è stata suddivisa in tre fasi incrementali di complessità.

3.1 Basic Networking e Interazione L2

Inizialmente è stata testata una topologia a switch singolo (`single,2`). L'uso di **netcat** ha permesso di validare il trasporto TCP tra host isolati.

Test Netcat

```
# h1 ascolta, h2 invia payload
h1# nc -l -p 5432
h2# nc 10.0.0.1 5432 "Hello Mininet"
```

3.2 Routing L3 Personalizzato

Tramite l'API Python di Mininet, è stato implementato un `GenericRouter`. Il nodo router abilita l'instradamento globale tramite il flag del kernel `ip`$_forward$:

3.3 Emulazione Vincoli Fisici

È stata applicata la classe `TCLink` per simulare un link a **100 Mbps** con un ritardo di **75ms** per ogni tratta, portando l'RTT complessivo tra host distanti a circa 300ms.

3.4 Istruzioni di Esecuzione

Le topologie Mininet devono essere avviate come root:

Mininet Run

```
$ sudo python3 ex2_router.py
$ sudo python3 ex3_tc.py
```

4 Risultati

La validazione è stata effettuata tramite tool di misura (`ping`, `iperf`) e analisi dei protocolli.

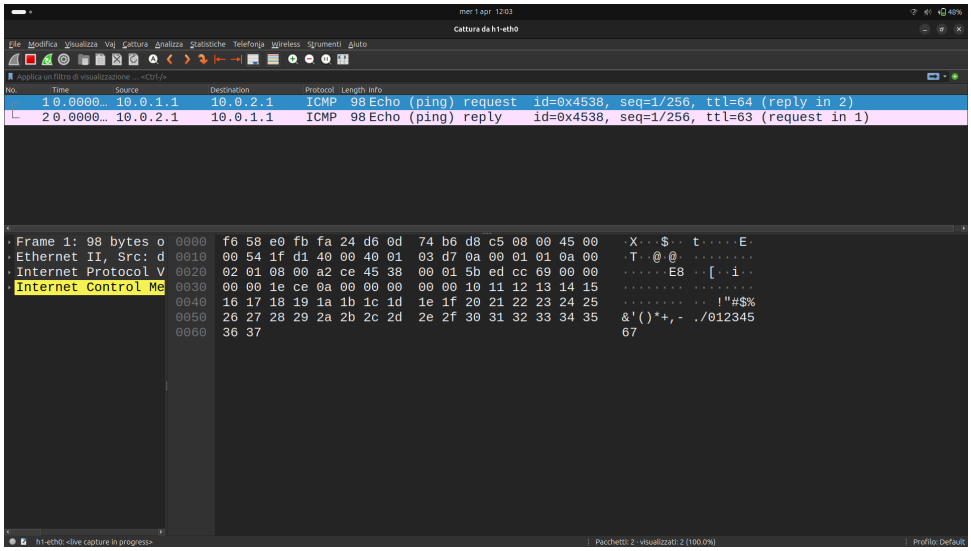

Figure 2: Analisi traffico SDN in topologia router-based (Esercizio 2).

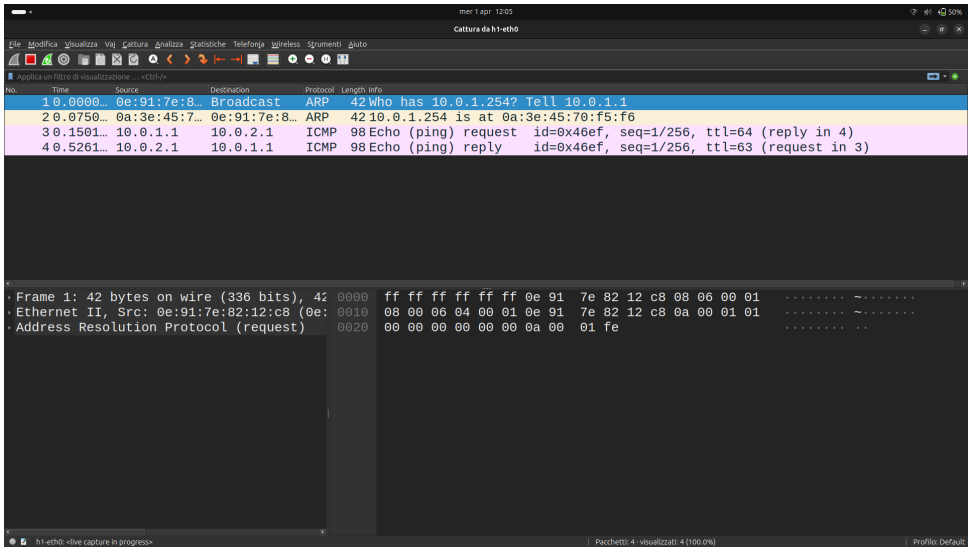

Figure 3: Verifica ritardi e banda con TC su link virtuale (Esercizio 3).

4.1 Analisi Wireshark

L'ispezione della cattura (Figura 7) evidenzia graficamente l'impatto dei vincoli: i tempi tra le richieste ARP e le relative risposte riflettono esattamente il delay configurato, confermando l'accuratezza della simulazione basata su `tc`.

4.2 Benchmark Network Constraints

La verifica dei vincoli fisici è stata effettuata tramite le utility `ping` e `iperf` integrate nella CLI di Mininet.

Test di Connettività e Banda

```
mininet> h1 ping -c 3 h2
64 bytes from 10.0.2.1: icmp_seq=1 ttl=63 time=300 ms
64 bytes from 10.0.2.1: icmp_seq=2 ttl=63 time=300 ms

mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['94.2 Mbits/sec', '95.1 Mbits/sec']
```

Il tempo di RTT misurato (300ms) corrisponde esattamente alla somma dei delay dei quattro attraversamenti dei link (75ms * 4), mentre il throughput si attesta vicino al limite dei 100 Mbps configurati, dimostrando la fedeltà del modello `TCLink`.

5 Conclusioni e Sviluppi Futuri

L'esperienza con Mininet ha dimostrato come la virtualizzazione permetta di testare architetture di rete complesse con costi nulli. L'integrazione tra script Python e kernel Linux rende SDN uno strumento potente per la prototipazione rapida.
Sviluppi futuri:

- **P4 Programming**: Evoluzione verso il linguaggio P4 per definire il comportamento del Data Plane in modo ancora più granulare rispetto a OpenFlow.
- **Network Slicing**: Implementazione di tecniche di isolamento delle risorse per creare "fette" di rete virtuali indipendenti sopra la stessa infrastruttura fisica, anticipando le architetture 5G.

to.